

What Makes an Exceptional AI-Assisted Programmer? Definitions, Skills, and a Research Agenda for CS Education

Author 1

Anonymous Institution
City, Country
anonymous@example.com

Author 2

Anonymous Institution
City, Country
anonymous@example.com

Author 3

Anonymous Institution
City, Country
anonymous@example.com

Abstract

AI coding agents bridge the gap between natural-language problem solving and machine-readable code. Because computer behavior can now be fully specified in natural language and translated into code through AI coding agents, we argue that specification skills, which have traditionally been learned through code implementation, must now be taught explicitly to enable students and professionals to write programs without mastering abstract programming languages. Prior work, however, conflates requirement engineering with prompt engineering, scenario-based testing, and probing software specifications, making it difficult to define precise learning practices for specification pedagogy. Requirement engineering and software testing have traditionally been framed as refinement and confirmatory activities, respectively, yet both resemble (though do not equal) the mechanisms for eliciting specifications from ill-specified requirements. This paper reviews these misconceptions, establishes canonical definitions and learning practices for AI-assisted programming, defines a new specification-based paradigm, and outlines future research and pedagogical agendas.

CCS Concepts

• **Social and professional topics** → **Computing education**; • **Software and its engineering** → *Requirements analysis*.

Keywords

Requirement Engineering, CS1 Education, AI-Assisted Programming, Test-Driven Learning, Exploratory Testing

ACM Reference Format:

Author 1, Author 2, and Author 3. 2026. What Makes an Exceptional AI-Assisted Programmer? Definitions, Skills, and a Research Agenda for CS Education. In *Proceedings of Proceedings of the 3rd ACM Virtual Global Computing Education Conference (SIGCSE Virtual '26)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

When discussing the implications of computers on education, Papert argued that computers enable "knowledge that was accessible only through formal processes" to be "approached concretely"

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGCSE Virtual '26, Virtual Event

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/12
<https://doi.org/XXXXXXX.XXXXXXX>

through the "Logo" programming language [37, p. 22]. While Papert viewed programming as a means of making math and other subjects more tangible (through computer simulations and programs), we view AI-assisted programming as a means of teaching computer programming without the need for precise and seemingly abstract coding languages, which introduce significant barriers to learning and constructing computer programs for non-majors [9].

AI-assisted programming enables developers to specify the system behavior using interfaces that are more natural to how humans communicate, such as through natural language, diagrams, whiteboard drawings [43], and screenshots [7]. Thus, it may no longer be necessary to require that students learn programming languages to construct programs, and we argue that CS education must adapt their curricula and learning objectives to reflect the abilities of AI coding assistants in building software systems from a set of specifications, which may not necessarily be (and are often not) written in a programming language [8, 13, 48].

State-of-the-art LLMs perform well at coding tasks, with Claude-Opus-4.5 solving 76.80%¹ of SWE-Bench problems [25]. Further, AI coding agents are becoming ubiquitous within academic institutions [39] and industry environments [14], meaning that AI-assisted programming may become a necessary skill for undergraduates applying to graduate school or for employment. Hence, an undergraduate education in computer science must include the necessary skills to engage in AI-assisted programming tasks.

Recent literature on AI-assisted programming has identified requirement engineering as a core skill in improving the quality of AI-generated code [2, 22, 32]. Our position is that AI-assisted programming research should shift its attention away from requirement engineering pedagogy and instead teach skills that are adjacent to *design*. We claim that in connecting requirement and prompt engineering, much of the recent literature either conflates the historical definition of requirement engineering with designing system specifications, or fails to ground AI-assisted programming education literature with previous definitions entirely [7, 22, 33, 40].

To better understand the differences in requirement engineering and designing specifications, we map our definition of *specification design* (section 4) to the attributes of an *exceptional designer*, used by Curtis et al. [10] to describe the system-level thinkers operationalize requirements into specifications. Thus, to facilitate the development of AI-assisted programming, we map the skills of an *exceptional AI-assisted programmer* onto a specification-driven process. To begin, we juxtapose the historical definition(s) of requirement engineering (subsection 2.1) with their current uses and misconceptions in AI-assisted programming literature (subsection 2.2). After refining these definitions, we use them to elaborate on the skills of an

¹<https://www.swebench.com/index.html>

exceptional AI-assisted programmer in section 3, which we decompose into abstraction traversal (subsection 3.1) and scenario-based testing (subsection 3.2), which we claim encompass specification articulation in subsection 3.3. Given the divergent definitions of requirement engineering presented in section 2, we propose referring to the process of eliciting and designing specifications for AI-assisted programming as *specification design* (section 4). Finally, we outline the implications of *specification design* on AI-assisted programming for research and pedagogy (section 5).

2 Specifying Software Requirements

The canonical definition of requirement engineering [58] is distinct from the use of requirement engineering terminology in AI-assisted programming contexts [32, 38, 56]. Crucially, historical requirement engineering involves different tasks (i.e. systematic elicitation), roles (i.e. stakeholders, project managers), and levels of abstraction (i.e. concerning abstract system objectives and constraints) than what is required for designing specifications for AI-assisted programming [12, 18, 44]. Conflating these two distinct tasks have measurable downstream pedagogical costs [2], and if we do not first reconcile these differences before analyzing the skills of an *exceptional AI-assisted programmer* and proposing future agendas, we risk failing to prepare students in writing complete and correct specifications for AI coding tools.

2.1 Historical Requirement Engineering

Requirement engineering has been defined as "the branch of software engineering concerned with the real-world goals for, functions of, and constraints on software systems" [58] in many seminal papers [35, 36, 50]. It specifically "includes learning, uncovering, surfacing, and discovering the needs (requirements) of the stakeholders of the intended system" [20]. Notably, key parts are omitted from this definition: while requirement engineering includes the assimilation, negotiation, and articulation of external goals for a software system, it does not involve designing a set of specifications that determine the realization of these goals [59]. In other words, the objective of requirement engineering is to specify stakeholder needs (*requirements*), but it fails to address the task of determining how a system should meet these needs (the *specifications* of the system). After the specifications have been articulated by the exceptional designer, a (or team of) software engineer(s) (SWE) write code in alignment with the specifications. At a high level, requirements 'express the desires of the customer concerning the software development project,' whereas specifications are 'intended to be directly implementable and to support satisfaction of the requirements [59].

Researchers have developed a set of normative requirement engineering frameworks, although they are seldom used in practice [36, 52]: GORE [49], scenario-based RE [46], PREView [44], and a plethora of adjacent techniques [35]. These frameworks primarily operate at the function level of the Structure Behavior and Function (SBF) Framework [18], a planning and design framework that introduces the following: the structure of a system is its components and their connections, the behavior is how the components interact, and the function is the system's high-level purpose (the functionality afforded by the system's behaviors). We make an important

departure from previous applications of the SBF Framework in computer science education literature, whereby instead of using code to represent the structure of a system [19], we extend a system's structure to its users, data streams, and more generally, its different components that comprise a composite system [49]. Requirements are also often developed and expressed at the function level, making domain knowledge (to understand the intended functionality) instrumental for requirement engineering [20].

It is crucial that we imbue a complete understanding of a system's behavior, how the structural elements interact to achieve the function [19], into AI-assisted programmers, which must obviously (due to the aforementioned definition of behavior) begin by developing a complete understanding of the structural components of the system. In the remainder of this section, we examine how traditional requirement engineering navigates this gap: first by surveying two paradigms that elicit requirements through goal- and scenario-based probing, and then by characterizing the *exceptional designer* whose tacit system-level knowledge bridges abstract requirements and concrete implementation with specifications.

2.1.1 Paradigms for Eliciting Requirements. To clearly delineate between traditional requirement engineering and the process of generating and disambiguating ill-defined specifications, as is necessary when prompting LLMs [38, 56], we analyze two seminal paradigms: Goal-Oriented Requirement Engineering (GORE) [49, 50] and the Scenario-Based Requirements Analysis Method (SCRAM) [45, 46]. GORE decomposes high-level system goals (e.g., serve more passengers") into progressively finer sub-goals through why" and "how" questioning, ending when a goal falls under the responsibility of a single agent and can therefore be expressed as a requirement [49]. Conversely, SCRAM elicits requirements by walking stakeholders through concrete interaction scenarios—often supported by early prototypes or storyboards—to surface reactions and tacit assumptions that abstract goal decomposition would leave implicit [46].

Despite these distinctions, "goal elaboration and scenario elaboration are intertwined processes," as Van Lamsweerde and Willemet assert that "a concrete scenario description may prompt the elicitation of the specifications of the goals underlying it" [51]. Both SCRAM and GORE presuppose that the requirements are known by the stakeholders (perhaps implicitly) and left unstated, treating requirement engineering as means of probing the requirements instead of defining the system behavior. Crucially, while their means of probing requirements differ, the deliverable is the same and requires SWEs to use domain and tacit knowledge to traverse between the functional specifications and their implementation, which is non-trivial for novices [21].

2.1.2 The Exceptional Designer. The traditional requirement engineering process involves the refinement of stakeholder goals, in contrast with generative requirement engineering. Refining stakeholder goals often occurs through *requirement negotiation*—resolving "conflicts between stakeholders without necessarily weakening satisfaction of each stakeholder's goals" [35]—whereas *generative requirement engineering* depends on probing the behavior and structure of the system [12, 32, 47]. To decompose the high-level requirements into coding tasks and design decisions, Curtis et al. introduces "exceptional designers," who are "adept at identifying unstated requirements, constraints, or exception conditions" [10].

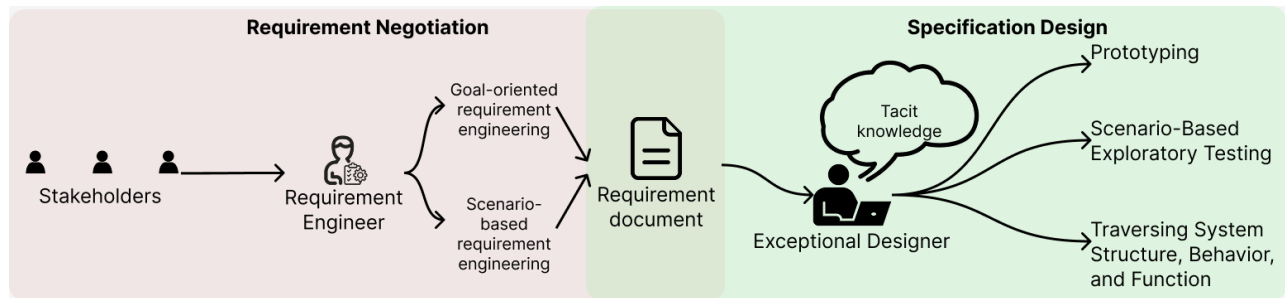


Figure 1: Demonstrating requirement-based software engineering through requirement negotiation and *specification design*.

Similar to how recent literature has used the term "requirement engineer" to refer to eliciting and articulating specifications [32], many of these designers focused on the design of system requirements, rather than code. Since experts are able to easily traverse between the SBF of a complex system [21], designers could reasonably derive concrete requirements about the code's implementation from capability-level specifications without implementation details through analyzing the structure and behavior of a system and connecting these observations to the specifications (Figure 1). However, this process comprises tacit computational thinking—"using abstraction and decomposition when attacking a large complex task or designing a large complex system" [54]—that designers have developed over years of practice; it is implicit and often learned through deep immersion in a problem space or domain [10].

2.2 Post-AI Requirement Engineering

Following the public release of ChatGPT in 2022, there was a surge of AI-assisted programming papers, many of which invoke the language of "requirements," "specifications," and "requirement engineering." However, much of this recent literature almost entirely forgoes the canonical RE definition of eliciting requirements from stakeholders. Of the AI-assisted programming papers we surveyed, only two papers [11, 57] cite the canonical requirement engineering lineage substantively, with the remaining papers omitting a formal definition for requirements and specifications [4, 6, 8, 13, 14, 25, 29, 39, 53] or conflating the two terms [12, 28, 33, 38, 47]. Additionally, some papers attempting to ground their work in previous specification and requirement engineering literature misinterpret the definition of requirement engineering. For example, Chen et al. define requirement engineering as "the foundation of software development, focusing on translating user needs into clear, actionable specifications," which is in direct contrast with the canonical definition of requirement engineering as a process of deriving acceptance criteria and constraints from stakeholders. The discrepancies in these two definitions represent the broader misunderstanding of requirement engineering for AI-assisted programming, thus motivating the distinction between designing specifications and requirement engineering.

Consider Robinson et al. [40] who combine requirements and design with code implementation because "with automation of implementation from requirements [with AI-assisted coding], the design and elicitation phase can include rapid prototyping to clarify user intentions." We note that according to the aforementioned definitions of requirement engineering and specification, Robinson

et al. are likely referring to system specifications when they invoke the term "requirements." Nevertheless, this iterative cycle matters because design refinement has typically occurred during implementation of code [5], so when the implementation is being delegated to an AI-assisted coding agent, the skill of designing specifications must be explicitly taught.

There have been some proposals to assess the identification of ambiguous specifications through probing a system's behavior [38], CS education research suggests that there is a pedagogical gap in teaching these probing activities, stating that "scaffolding students in the process of specifying effective questions may be a useful approach during future iterations," [47]. We argue that when literature oversimplifies the skills needed for AI-assisted programming by connecting them to requirement engineering, it inadvertently alters the learning objectives and necessary skills associated with specifying system behavior. Similarly, when authors assert the necessity of requirement engineering without fully understanding previous literature, it causes confusion in the CS education community over the usage of requirement engineering.

When literature oversimplifies the skills needed for AI-assisted programming by connecting them to requirement engineering, it inadvertently alters the learning objectives and necessary skills associated with specifying system behavior, and when authors assert the necessity of requirement engineering without fully understanding previous literature, it causes confusion in the CS education community. There are clearly divergent definitions of requirement engineering in AI-assisted programming literature, necessitating the creation of an unambiguous specification-oriented design process specifically for AI-assisted programming.

3 Exceptional AI-Assisted Programmers

Curtis et al. describe exceptional designers as interdisciplinary [10], enabling them to bridge the gap between stakeholders and programmers who lack domain expertise. We treat the *exceptional AI-assisted programmer* as a direct analog who operates at a higher level of abstraction in comparison to a traditional programmer. The definitions of and connections between requirement engineering, design, and specifications are often divergent and inconsistent [1, 16, 24, 41, 46, 49]. This makes it difficult to define the learning objectives of a curriculum to support students in AI-assisted programming. We argue that discovering and articulating specifications—a core skill for AI-assisted programming [32]—requires different pedagogy than requirement engineering.

3.1 Traversing Structure, Behavior, and Function

Batista et al. [2] taught the GORE framework in a novice CS course for AI requirement engineering. Among students with a rudimentary understanding of GORE, 100% identified users and features, but only 53.33% identified user-data behavior and 43.33% explained system functionalities. GORE thus aids identification of system function but not the behavior or structure of the system, which we have shown are integral to designing specifications for AI-assisted programming. This aligns with empirical results on SBF traversal [21]: novices grasp structure but struggle with behavior and function. We posit that novices' lack of understanding of system behavior and functionality may stem from their inability to traverse between structure, behavior and function, which GORE and similar frameworks do not explicitly teach. When designing a system, exceptional designers "map between the behavior required of the application system and the computational structures that implemented this behavior" [10]. Similarly, we believe exceptional AI-assisted programmers should be able to traverse between the structure, behavior, and function of their system by using domain information to inform system-level thinking.

Although prompting for AI-assisted coding remains a "primarily self-taught process with 'trial-and-error'," [23], professional developers stated the importance of their existing SWE expertise "as a critical strategy for effectively using [AI coding] agents when creating software." This supports the necessity of traversing the structure, behavior, and function of a system because the experts relied on their tacit knowledge to translate project specifications into code using AI coding agents, which is precisely the role of an *exceptional AI-assisted programmer*.

3.2 Scenario-Based Exploratory Testing

The design of interactive systems are ill-defined and are often solved using "solution-first" approaches instead of scenario-driven problem solving, which diminishes the quality of the specifications [41]. In Behavior-Driven Development (BDD), designers receive a set of high-level requirements (e.g. A banker must be able to withdraw money) and create "given-when-then" scenarios to "de-scope some of the more esoteric requirements" [34], which again we refer to as specifications instead of requirements. Due to the exploratory nature of designing specifications, it is important that AI-assisted programmers iteratively develop end-user scenarios to enlighten specifications. There has been success thus far in using LLM-generated test cases to clarify ambiguities in natural language prompts [17], and we see scenario-based testing to generate new specifications as a natural extension of this paradigm.

We note that software testing is often considered independently of requirement engineering in educational contexts [11], with very little literature analyzing the use of software testing in discovering requirements. However, previous literature demonstrates that "problem and program comprehension strategies and abilities appear to be intricately tied to students' abilities to devise useful test cases," which informs our "requirement-first" approach to testing [42]. Since students predominantly write tests that fail to adequately explore boundary values [15], testing for requirements with edge cases is particularly helpful in avoiding the "happy path" bias.

Scenario-based boundary testing is also used in GORE to uncover missing requirements [50].

To further illustrate the connection between testing and requirement engineering, we consider Problem-Oriented Programming and Testing (POPT), which uses ill-defined specifications to challenge a student's testing skills [31]. POPT requires students to define test cases through input-output pairs before and during code development, similar to Test-Driven Development [3] and exploratory testing [1]. These strategies "help the student to enlighten the problems' specifications," [31] which is precisely the skill we hope to develop for *exceptional AI-assisted programmers*. A related paradigm, *probeable problems* [38], tasks students with probing deliberately under-specified problems to uncover hidden requirements. However, since it targets competitive programming, it does not require students to engage with more open-ended domains in which requirements are more ambiguous. A gap therefore remains in the literature on pedagogy for teaching these requirement-revealing testing skills, as discussed in subsection 2.1.

3.3 Specification for AI-Assisted Coding

Unambiguously expressing a system's specifications is particularly germane to AI-assisted programming because underspecification increases misalignment between the developer's intention and the code output; two studies report that the underspecified prompt is considerably less aligned with the developer's (often implicit) acceptance criteria than the fully-specified prompt [28, 56]. This indicates that not only do students need to design specifications, but they also must accurately and comprehensively describe them.

Exceptional designers "spent much of their time educating others about the application domain and its mapping into computational structures" [10]. Similarly, we view the role of an AI-assisted programmer as an educator, whereby the prompter must articulate requirements to the LLM via a prompt. Articulating unambiguous and complete specifications to LLM coding agents is both crucial and non-trivial; specification ambiguities reduce accuracy by roughly 30%, while omissions reduce accuracy by roughly 25% [28]. This result is corroborated by an additional study proposing "traditional requirement engineering approaches" in prompts to minimize underspecification [56].

Given these costs, future pedagogy must reconsider what specification skill it emphasizes. Instead of strictly teaching students to articulate implicit requirements [32, 55], which typically requires the student to understand the behavior of the system, which is not a natural skill for novices [21], future pedagogy should strongly consider teaching students to create and evaluate scenarios to aid in designing specifications. Regardless, it is obvious that accurate and complete specifications are crucial for AI-assisted programming tasks, driving recent literature to draw parallels between prompt engineering and requirement engineering [22, 32, 47].

Recent literature [13, 30, 53] on prompt engineering pedagogy often fails to teach students to derive specifications for ill-defined problems, one of the core skills for AI-assisted programming. Indeed, it is considered that the "central bottleneck" to writing adequate prompts is validating specifications and formalizing intent [27], which ignores the necessity of discovering specifications for ambiguous problems. In fact, in one survey [23] on professional

Table 1: Our reframing of the Structure, Behavior, and Function (SBF) framework [18] for AI-assisted programming, contrasted with its previous usage in CS education literature.

SBF	Previous Usage	Our Definition
Structure	Lines of code	Users, data, interfaces
Behavior	Inputs and outputs	The interaction of structural components
Function	Acceptance criteria	Requirements

developers interacting with AI coding agents, "most participants agreed that prompts should include clear context and explicit instructions," which corroborates our observation of the importance of clearly specifying and articulating system specifications for AI-assisted programming. Thus, by surveying previous literature analyzing the behavior of professional software developers, *we argue that AI-assisted programming pedagogy must primarily concern itself with designing specifications, rather than requirement engineering.*

4 Introducing "Specification Design"

Given the discrepancies in how requirement engineering has been used in the past and its current implications in AI-assisted programming, we advocate for the transition away from the comparison between requirement engineering and prompt engineering. Both to motivate this transition and to better understand the necessary skills for *exceptional AI-assisted programmers*, we must first acknowledge the distinction between requirements and specifications. Zave & Jackson [59] assert that while "requirements engineering is about the satisfaction of [stakeholder] goals," they must only describe "what a machine does, not how it does it." This is an important distinction, because as we saw above, *exceptional designers* must often translate abstract requirements into concrete coding plans, which seems orthogonal to this definition of requirement engineering. Indeed, we see that professional developers do not simply state the "optative goals" (representing the idealistic environment with the inclusion of the computer program), but they also "controlled the design of new features" and "controlled the software implementation at some level," [23]. Clearly, these behaviors appear to be more reflective of "designing" specifications than requirement engineering.

To more accurately reflect the tasks of an *exceptional AI-assisted programmer* and to prevent confusion with requirement-engineering adjacent terms, such as "the refinement of requirements," [59], we introduce the term **specification design**.

Specification Design. *The process of using a system's structure to probe and iteratively refine specifications based on the behavior of the system.*

Here, *structure* denotes the system's users, data streams, and interfaces, and *behavior* the scenarios describing how those components interact (see Table 1). Specification design is therefore inherently iterative and behavior-driven. Rather than articulating a complete specification upfront, the AI-assisted programmer treats

structure as a means of surfacing behavioral cases that in turn reveal omitted or ambiguous specifications. This reframing carries direct consequences for what we teach and study—shifting attention from requirement elicitation toward the skills of traversal, scenario design, and articulation that address in the following section.

5 Implications for Research and Pedagogy

We conclude by posing the question: *what skills should an exceptional AI-assisted programmer possess?* In this section, we decompose AI-assisted programming into three complementary skills, explore their motivations, and expand upon their implications for research and pedagogy. We map the three unique attributes of exceptional designers [10] onto skills for AI-assisted programming.

5.1 How do Novices Perform at Abstraction?

Novices struggle with understanding the behavior and function of a system [21], but to better understand their weaknesses as they relate to AI-assisted programming, we must examine reasoning at all three levels of abstraction while engaging in *specification design*. We propose modeling and designing interventions around how *exceptional AI-assisted programmers* traverse between levels and use the system's behavior to inform specifications.

5.1.1 Research: comparing expert and novice SBF traversal. Future research must investigate how AI-assisted programmers traverse between the structure, behavior, and function of a system, and should empirically validate interventions that develop the student's understanding of system behavior from structure. We advocate for investigations that compare the AI-assisted programming practices of experts and novices, particularly using a think-aloud protocol to understand the metacognition and problem-solving approaches of each population and inform further work in how to scaffold the process of transforming novice practices into expert ones.

5.1.2 Pedagogy: teaching traversal between structure, behavior, and function. Since GORE and similar frameworks aid in the identification of system function but do not explicitly teach behavior or structure, future pedagogy should explicitly scaffold traversal between the structure, behavior, and function of a system. We see the incorporation of *specification design training* into computer science courses as a means of increasing the role of domain knowledge in software development and reducing the gap between the developer and the end-user.

5.2 How do Experts and Novices Probe Scenarios?

Given that scenarios are crucial to understanding the specifications of a system [41], we must identify the tacit knowledge that experts rely on to develop test cases to probe specifications. We must use these observations to inform testing scaffolds, hint delivery, and more broadly, the design of scenario-based curricula.

5.2.1 Research: how developers write scenario-based tests. Future research must further explore the use of scenario-based exploratory testing to teach specification design to novice AI-assisted programmers. We call for research exploring how developers write scenario-based tests in plain English, and how they draw upon past software

development experience. Further, future work must investigate the help-seeking behaviors of this population, and use those findings to develop a pedagogical system that teaches students to engage in scenario-based testing for specification design.

5.2.2 Pedagogy: formal learning objectives for specification design. We propose teaching behavior-based exploratory testing to facilitate iterative scenario design. Through formative studies and experience reports, we hope to see the emergence of formal learning objectives for specification design. We would also be delighted to see the evolution of the specification design paradigm into a formal, more systematic process with measurable learning objectives, pre-requisites, assessment strategies, and novel instructional methodologies, ultimately developing future *exceptional AI-assisted programmers*.

5.3 How do Novices Articulate Specifications?

The ability to precisely articulate a system's specifications during and following *specification design* is clearly an important skill, as experienced AI-assisted programmers "control the software design and implementation by prompting and planning with clear context and explicit instructions" [23]. Thus, we must investigate what enables these designers to clearly articulate requirements, and design appropriate pedagogical scaffolds around these skills.

5.3.1 Research: specification design for non-traditional programmers. We are excited about the possibilities of AI-assisted programming to increase the accessibility of computer programming to non-CS majors, conversational programmers, and end-user programmers. Conversational programmers, who wish to develop a fluency of computing terminology to communicate with technical colleagues, report feeling discouraged after taking traditional syntax-heavy courses because their end goal isn't a career in programming [9]. End-user programmers, on the other hand, view programming as a means to an end, whereby they wish to use their programming expertise to build projects that assist them in their everyday tasks [26]. We see a future where the technical and motivational gap between these two groups converge, and we propose future research that theorizes the effect of AI-assisted programming on these two increasingly large populations.

5.3.2 Pedagogy: low-code instruction in introductory courses. We find it important that introductory computing courses, which have large numbers of students who are not majoring in computer science, experiment with low-code or no-code assignments and instruction that instead focuses on training students to articulate complete specifications, free of omission and commission errors, when prompting LLMs. Teaching students to be *exceptional AI-assisted programmers* by teaching *specification design* eliminates barriers to computer programming and increases the role of domain knowledge in software development.

6 Conclusion

In this paper, we highlight the misassociation between AI-assisted programming and requirement engineering and introduce *specification design* and a future agenda to help students become *exceptional AI-assisted programmers*. We map the skills of an exceptional AI-assisted programmer the *exceptional designer* [10]. We claim that

the skills of an exceptional AI-assisted programmer include: (1) traversing multiple levels of abstraction across a system's structure, behavior, and function [18], (2) designing scenario-based tests that highlight the interaction of different structural components, and (3) clearly articulating the specifications that are derived from skills 1 and 2. The actionable agenda in section 5 and new definition of *specification design* can be used to conduct targeted empirical research and surveys into the specification and scenario-based testing behavior of novices and experts in AI-assisted programming, as well as in the design of future pedagogical interventions.

Acknowledgments

This sentence represents one acknowledgment and is anonymous for review.

References

- [1] James Bach. 2003. Exploratory testing explained.
- [2] Beatriz Braga Batista, Márcia Sampaio Lima, and Tayana Uchôa Conte. 2024. Teaching Requirements Engineering for AI: A Goal-Oriented Approach in Software Engineering Courses. In *Proceedings of the XXIII Brazilian Symposium on Software Quality*. ACM. doi:10.1145/3701625.3701686
- [3] Kent Beck. 2003. *Test-driven development: by example*. Addison-Wesley Professional.
- [4] Brett A. Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. Programming Is Hard – Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*. ACM, 500–506. doi:10.1145/3545945.3569759
- [5] Frederick Brooks and H Kugler. 1987. *No silver bullet*. April.
- [6] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. 2025. Unleashing the potential of prompt engineering for large language models. *Patterns* 6, 6 (2025).
- [7] Zhenpeng Chen, Chong Wang, Weisong Sun, Xuanzhe Liu, Jie M Zhang, and Yang Liu. 2026. Promptware Engineering: Software Engineering for Prompt-Enabled Systems. *ACM Transactions on Software Engineering and Methodology* (2026).
- [8] Bruno Pereira Cipriano and Lúcio Studer Ferreira. 2025. Programmers Aren't Obsolete Yet: A Syllabus for Teaching CS Students to Responsibly Use Large Language Models for Code Generation. arXiv:2502.15493 [cs.SE] <https://arxiv.org/abs/2502.15493>
- [9] Kathryn Cunningham, Julie Blanchard, Barbara Ericson, and Mark Guzdial. 2021. Avoiding the Turing Tarpit: Learning Conversational Programming by Starting from Code's Purpose. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM. doi:10.1145/3411764.3445571
- [10] Bill Curtis, Herb Krasner, and Neil Iscoe. 1988. A field study of the software design process for large systems. *Commun. ACM* 31, 11 (1988), 1268–1287.
- [11] Marian Daun, Alicia M. Grubb, Viktoria Stenkova, and Bastian Tenbergen. 2023. A Systematic Literature Review of Requirements Engineering Education. *Requirements Engineering* 28 (2023), 145–175. doi:10.1007/s00766-022-00381-9
- [12] Paul Denny, Viraj Kumar, Stephen MacNeil, James Prather, and Juho Leinonen. 2025. Probing the unknown: Exploring student interactions with probable problems at scale in introductory programming. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 618–624.
- [13] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. 2024. Prompt Problems: A New Programming Exercise for the Generative AI Era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. ACM, Portland OR USA, 296–302. doi:10.1145/3626252.3630909
- [14] DORA. 2025. *2025 State of AI-assisted Software Development*. Annual Research Report. Google Cloud. https://services.google.com/fh/files/misc/2025_state_of_ai_assisted_software_development.pdf Version 2025.2.
- [15] Stephen H. Edwards and Zalia Shams. 2014. Do Student Programmers All Tend to Write the Same Software Tests?. In *Proceedings of the 2014 Conference on Innovation and Technology in Computer Science Education*. 171–176. doi:10.1145/2591708.2591757
- [16] Eduard Paul Enoiu, Gerald Tukseferi, and Robert Feldt. 2020. Towards a Model of Testers' Cognitive Processes: Software Testing as a Problem Solving Approach. In *IEEE International Conference on Software Quality, Reliability and Security Companion*. 272–279. doi:10.1109/QRSC-51114.2020.00053
- [17] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User

- study and empirical evaluation. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2254–2268.
- [18] Ashok K. Goel, Spencer Rugaber, and Swaroop Vattam. 2009. Structure, behavior, and function of complex systems: The structure, behavior, and function modeling language. *Artificial Intelligence for Engineering Design, Analysis and Manufacturing* 23, 1 (2009), 23–35. doi:10.1017/S0890060409000080
 - [19] Mark Guzdial. 2018. *Teaching to Develop a Mental Model of Program Behavior: How Do Students Learn the Notional Machine*. Computing Education Research – Guzdial’s Take. <https://computingeducation.wordpress.com/2018/04/06/how-do-students-learn-the-notional-machine-developing-a-mental-model-of-program-behavior/>
 - [20] Irit Hadar, Pnina Soffer, and Keren Kenzi. 2014. The role of domain knowledge in requirements elicitation via interviews: an exploratory study. *Requirements Engineering* 19, 2 (2014), 143–159.
 - [21] Cindy E. Hmelo-Silver and Merav Green Pfeffer. 2004. Comparing Expert and Novice Understanding of a Complex System from the Perspective of Structures, Behaviors, and Functions. *Cognitive Science* 28 (2004), 127–138. doi:10.1207/s15516709cog2801_7
 - [22] Kaicheng Huang, Fanyu Wang, Yutan Huang, and Chetan Arora. 2025. Prompt engineering for requirements engineering: A literature review and roadmap. In *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*. IEEE, 548–557.
 - [23] Ruanqianqian Huang, Avery Reyna, Sorin Lerner, Haijun Xia, and Brian Hempel. 2025. Professional Software Developers Don’t Vibe, They Control: AI Agent Use for Coding in 2025. *arXiv preprint arXiv:2512.14012* (2025).
 - [24] Juha Itkonen, Mikka V. Mantyla, and Casper Lassenius. 2009. How do testers do it? An exploratory study on manual testing practices. In *2009 3rd International symposium on empirical software engineering and measurement*. IEEE, 494–497.
 - [25] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Xexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
 - [26] Andrew J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. 2011. The State of the Art in End-User Software Engineering. *Comput. Surveys* 43, 3 (2011), 1–44. doi:10.1145/1922649.1922658
 - [27] Shuvendu K. Lahiri. 2026. Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents. *arXiv preprint arXiv:2603.17150* (2026).
 - [28] Maya Larbi, Amal Akli, Mike Papadakis, Rihab Bouyousfi, Maxime Cordy, Federica Sarro, and Yves Le Traon. 2025. When prompts go wrong: Evaluating code model robustness to ambiguous, contradictory, and incomplete task descriptions. *arXiv preprint arXiv:2507.20439* (2025).
 - [29] Jenny T. Liang, Chenyang Yang, Agnia Sergeyuk, Travis D. Breaux, and Brad A. Myers. 2024. Understanding Prompt Programming Tasks and Questions. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering*.
 - [30] Leo S. Lo. 2023. The CLEAR path: A framework for enhancing information literacy through prompt engineering. *The Journal of Academic Librarianship* 49, 4 (2023), 102720.
 - [31] Vicente Lustosa Neto, Roberta Coelho, Larissa Soares Leite, Dalton S. Guerrero, and Andréa Pereira Mendonça. 2013. POPT: A Problem-Oriented Programming and Testing Approach for Novice Students. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. IEEE, 1099–1108. doi:10.1109/ICSE.2013.6606660
 - [32] Qianou Ma, Weirui Peng, Chenyang Yang, Hua Shen, Kenneth R. Koedinger, and Tongshuang Wu. 2025. What Should We Engineer in Prompts? Training Humans in Requirement-Driven LLM Use. *ACM Transactions on Computer-Human Interaction* (2025). doi:10.1145/3731756
 - [33] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yang, Binqian Li, Chenxue Wang, Shichao Zhang, and Qing Wang. 2024. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024). doi:10.1145/3660810
 - [34] Dan North. 2006. Introducing BDD. *Better Software Magazine*. <https://dannorth.net/blog/introducing-bdd/>
 - [35] Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements Engineering: A Roadmap. In *Proceedings of the Conference on The Future of Software Engineering*. ACM, 35–46. doi:10.1145/336512.336523
 - [36] Barbara Paech, Tom Koenig, Lars Borner, and Aybüke Aurum. 2005. *An Analysis of Empirical Requirements Engineering Survey Data*. Springer Berlin Heidelberg, Berlin, Heidelberg, 427–452. doi:10.1007/3-540-28244-0_19
 - [37] Seymour Papert. 1980. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, Inc., New York, NY, USA.
 - [38] Mrigank Pawagi and Viraj Kumar. 2024. Probeable problems for beginner-level programming-with-AI contests. In *Proceedings of the 2024 ACM Conference on International Computing Education Research-Volume 1*. 166–176.
 - [39] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromír Šavelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*. ACM, 108–159. doi:10.1145/3623762.3633499
 - [40] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements are all you need: The final frontier for end-user software engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–22.
 - [41] Mary Beth Rosson and John M. Carroll. 2007. Scenario-based design. In *The human-computer interaction handbook*. CRC Press, 1067–1086.
 - [42] Austin M. Shin and Ayaan M. Kazerouni. 2024. A model of how students engineer test cases with feedback. *ACM Transactions on Computing Education* 24, 1 (2024), 1–31.
 - [43] David H. Smith, S. Moonwara A. Monisha, Annapurna Vadaparty, Leo Porter, and Daniel Zingaro. 2026. Prompting through Decomposition: Evaluating the Efficacy of Problem Decomposition Diagrams for Code Generation. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2 (USA) (SIGCSE TS 2026)*. Association for Computing Machinery, New York, NY, USA, 1525–1526. doi:10.1145/3770761.3771189
 - [44] Ian Sommerville and Pete Sawyer. 1997. Viewpoints: Principles, Problems and a Practical Approach to Requirements Engineering. *Annals of Software Engineering* 3 (1997), 101–130.
 - [45] Alistair Sutcliffe. 1998. Scenario-based requirements analysis. *Requirements engineering* 3, 1 (1998), 48–65.
 - [46] Alistair Sutcliffe. 2003. Scenario-based requirements engineering. In *Proceedings. 11th IEEE International Requirements Engineering Conference, 2003*. IEEE, 320–329.
 - [47] Kay Tang, Hoanh Nguyen Hosea Tong-Ho, Kaitlin Riegel, Paul Denny, Nasser Giacaman, and Juho Leinonen. 2026. Ambiguity by Design: Practicing Requirement Clarification through Natural-Language Dialogue with LLMs. In *Proceedings of the 28th Australasian Computing Education Conference*. 138–147.
 - [48] Annapurna Vadaparty, Daniel Zingaro, David H. Smith IV, Mounika Padala, Christine Alvarado, Jamie Gorson Benario, and Leo Porter. 2024. CS1-LLM: Integrating LLMs into CS1 Instruction. In *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education*. doi:10.1145/3649217.3653584
 - [49] Axel Van Lamsweerde et al. 2001. Goal-oriented requirements engineering: A guided tour. *re* 1 (2001), 249.
 - [50] Axel Van Lamsweerde and Emmanuel Letier. 2000. Handling obstacles in goal-oriented requirements engineering. *IEEE Transactions on software engineering* 26, 10 (2000), 978–1005.
 - [51] Axel Van Lamsweerde and Laurent Willemet. 1998. Inferring declarative requirements specifications from operational scenarios. *IEEE Transactions on Software Engineering* 24, 12 (1998), 1089–1114.
 - [52] Stefan Wagner, Daniel Méndez Fernández, Michael Felderer, Antonio Vetrò, Marcos Kalinowski, Roel Wieringa, Dietmar Pfahl, Tayana Conte, Marie-Therese Christiansson, Desmond Greer, et al. 2019. Status quo in requirements engineering: A theory and a global family of surveys. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 2 (2019), 1–48.
 - [53] Tianyu Wang, Nianjun Zhou, and Zhixiong Chen. 2024. Enhancing Computer Programming Education with LLMs: A Study on Effective Prompt Engineering for Python Code Generation. *arXiv preprint arXiv:2407.05437* (2024). doi:10.48550/arXiv.2407.05437
 - [54] Jeannette M. Wing. 2006. Computational Thinking. *Commun. ACM* 49, 3 (2006), 33–35. doi:10.1145/1118178.1118215
 - [55] Ruiwei Xiao, Xinying Hou, Runlong Ye, Majeed Kazemitabaar, Nicholas Diana, Michael Liut, and John Stamper. 2025. Improving student-ai interaction through pedagogical prompting: An example in computer science education. *arXiv preprint arXiv:2506.19107* (2025).
 - [56] Chenyang Yang, Yike Shi, Qianou Ma, Michael Xieyang Liu, Christian Kästner, and Tongshuang Wu. 2024. What Prompts Don’t Say: Understanding and Managing Underspecification in LLM Prompts. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM.
 - [57] Mohammad Amin Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. 2025. Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. *arXiv:2509.11446 [cs.SE]* <https://arxiv.org/abs/2509.11446>
 - [58] Pamela Zave. 1997. Classification of research efforts in requirements engineering. *ACM Comput. Surv.* 29, 4 (Dec. 1997), 315–321. doi:10.1145/267580.267581
 - [59] Pamela Zave and Michael Jackson. 1997. Four dark corners of requirements engineering. *ACM transactions on Software Engineering and Methodology (TOSEM)* 6, 1 (1997), 1–30.